

សាកលវិទ្យាល័យអង្គរ

ANGKOR UNIVERSITY

បរិញ្ញាបត្រ វិទ្យាសាស្ត្រកុំព្យូទ័រ

ឯកសារណែនាំ (Handout) - មគ្គុទ្ទេសក៍អនុវត្តប្រព័ន្ធសុវត្ថិភាព Dual-Auth

ASP.NET Core EF Core Code-First Approach
(MVC + RESTful Web API + Identity + JWT Token)

ការអភិវឌ្ឍកម្មវិធី Web និង Web API ជាមួយប្រព័ន្ធសុវត្ថិភាពទិន្នន័យ Cookie & JWT



រៀបចំដោយសាស្ត្រាចារ្យ៖

សាស្ត្រាចារ្យ សៀង លីម (Soeurng LIM)

☰ មាតិកាមគ្គុទ្ទេសក៍ (Guide Contents)

ជំហាន	ខ្លឹមសារអនុវត្ត (Topics & Steps)	Bloom's Level
ជំហានទី ០	ការរៀបចំ Project Setup និងការដំឡើង NuGet Packages ចាំបាច់	1. REMEMBER
ជំហានទី ១	ការបង្កើត Entity Models (`AppUser`) និង EF Core `ApplicationDbContext` (Code-First)	3. APPLY
ជំហានទី ២	ការបង្កើត Data Transfer Objects (DTOs) សម្រាប់ Login និង Token Response	3. APPLY
ជំហានទី ៣	ការអនុវត្ត Repository Pattern ដោយប្រើប្រាស់ EF Core Async Methods	3. APPLY
ជំហានទី ៤	ការកំណត់អត្ថបទកូដ Configuration នៅក្នុង `appsettings.json`	2. UNDERSTAND
ជំហានទី ៥	ការកំណត់ `Program.cs` សម្រាប់ Dual Auth (Cookie + JWT), Swagger និង Seeding	4. ANALYZE
ជំហានទី ៦	ការដំណើរការ EF Core Database Migrations ដើម្បីបង្កើត Database ស្វ័យប្រវត្ត	3. APPLY
ជំហានទី ៧	ការសរសេរ Web API Controllers (Auth API ជាមួយ JWT និង AppUser API CRUD)	6. CREATE
ជំហានទី ៨	ការសរសេរ MVC Controllers (Account Controller និង AppUser Controller)	6. CREATE
ជំហានទី ៩	ការបង្កើត Razor Views (.cshtml) សម្រាប់ទំព័រ UI របស់ MVC	6. CREATE
ជំហានទី ១០	ការវិភាគប្រៀបធៀប Cookie-Auth VS JWT-Bearer Auth & Security Defense	4. ANALYZE
ជំហានទី ១១	ការធ្វើតេស្ត និងការផ្ទៀងផ្ទាត់ប្រព័ន្ធ (Swagger, Postman, Web Views, SweetAlert2)	5. EVALUATE

🎯 វត្ថុបំណងនៃការរៀនសូត្រ (Bloom's Objectives)

- 1. REMEMBER EF Core Concepts:** ចងចាំច្បាស់ពីការបង្កើត Database/Tables ចេញពី C# Models ដោយប្រើ EF Core Code-First Migrations។
- 2. UNDERSTAND Dual Auth Flow:** យល់ដឹងពីភាពខុសគ្នារវាង **Cookie-Based Authentication** (សម្រាប់ MVC SSR) និង **JWT Bearer Token** (សម្រាប់ Stateless Web APIs)។
- 3. APPLY Repository & Async:** ចេះអនុវត្ត Repository Pattern និងប្រើ async/await ជាមួយ Entity Framework Core ក្នុងការគ្រប់គ្រង Database Operations។
- 4. ANALYZE Security Mechanics:** វិភាគនិងកំណត់រចនាសម្ព័ន្ធ Middleware ក្នុង `Program.cs` ដើម្បីការពារកុំឱ្យមានការវាយប្រហារ CSRF, XSS និង Unauthorized Access។
- 5. EVALUATE System Testing:** ធ្វើការវាយតម្លៃនិងធ្វើតេស្តប្រព័ន្ធ Auth ទាំងពីរតាមរយៈ Swagger UI, Postman Header Request និង Browser MVC Views។

❄ ជំហានទី ០: ការរៀបចំ Project និង NuGet Packages

1. REMEMBER

បង្កើត Project ថ្មីប្រភេទ ASP.NET Core Web App (MVC) និងដំឡើង NuGet Packages ចាំបាច់សម្រាប់ EF Core, ASP.NET Core Identity, JWT Authentication, និង Swagger UI៖

```
dotnet new mvc -n MyCodeFirstProject
cd MyCodeFirstProject

# ដំឡើង NuGet Packages សម្រាប់ EF Core Code-First, Identity, JWT & Swagger UI
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

🗄 ជំហានទី ១: Domain Entity Models & DbContext

3. APPLY

១.១ File: `Models/AppUser.cs` (Domain Entity)

```
using System.ComponentModel.DataAnnotations;

namespace MyCodeFirstProject.Models
{
    public class AppUser
    {
        [Key]
        public int Id { get; set; }

        [Required(ErrorMessage = "សូមបញ្ចូលឈ្មោះពេញ")]
        [StringLength(100)]
        [Display(Name = "ឈ្មោះពេញ")]
        public string FullName { get; set; } = "";

        [Required(ErrorMessage = "សូមបញ្ចូល Email")]
        [EmailAddress]
    }
}
```

១.២ File: `Data/ApplicationDbContext.cs` (Identity + Code-First)

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using MyCodeFirstProject.Models;

namespace MyCodeFirstProject.Data
{
    public class ApplicationDbContext : IdentityDbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options) { }

        // Code-First DbSet សម្រាប់តារាង AppUsers
        public DbSet<AppUser> AppUsers { get; set; }

        protected override void OnModelCreating(ModelBuilder builder)
        {
            // ...
        }
    }
}
```

➡ ជំហានទី ២: Data Transfer Objects (DTOs)

3. APPLY

`Models/DTOs/LoginDto.cs`

```
public class LoginDto
{
    [Required, EmailAddress]
    public string Email { get; set; } = "";

    [Required, DataType(DataType.Password)]
    public string Password { get; set; } = "";

    public bool RememberMe { get; set; }
}
```

`Models/DTOs/TokenResponseDto.cs`

```
public class TokenResponseDto
{
    public string Token { get; set; } = "";
    public DateTime Expiration { get; set; }
}
```

📦 ជំហានទី ៣: Repository Pattern (EF Core Async)

3. APPLY

៣.១ Interface: `Interfaces/IAppUserRepository.cs`

Interfaces/IAppUserRepository.cs

```
public interface IAppUserRepository
{
    Task<IEnumerable<AppUser>> GetAllAsync();
    Task<AppUser?> GetByIdAsync(int id);
    Task InsertAsync(AppUser user);
    Task UpdateAsync(AppUser user);
}
```

៣.២ Concrete Repository: `Repositories/AppUserRepository.cs`

Repositories/AppUserRepository.cs

```
public class AppUserRepository : IAppUserRepository
{
    private readonly ApplicationDbContext _context;
    public AppUserRepository(ApplicationDbContext context) => _context = context;

    public async Task<IEnumerable<AppUser>> GetAllAsync()
    => await _context.AppUsers.OrderByDescending(u => u.Id).ToListAsync();

    public async Task<AppUser?> GetByIdAsync(int id)
    => await _context.AppUsers.FindAsync(id);

    public async Task InsertAsync(AppUser user)
    {
        await _context.AppUsers.AddAsync(user);
        await _context.SaveChangesAsync();
    }

    public async Task UpdateAsync(AppUser user)
    {
        _context.AppUsers.Update(user);
        await _context.SaveChangesAsync();
    }
}
```

🔧 ជំហានទី ៤: Configuration ក្នុង `appsettings.json`

2. UNDERSTAND

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=UserManagementCodeFirstDB;Trusted_Connection=True",
  },
  "Jwt": {
    "Key": "YOUR_VERY_SECRET_KEY_HERE_32_CHARS_OR_MORE_123456",
    "Issuer": "https://localhost:7123",
    "Audience": "https://localhost:7123"
  }
}
```

</> ជំហានទី ៥: Program.cs (Dual Auth & Dependency Injection)

4. ANALYZE

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// 1. EF Core DbContext & Identity Services
builder.Services.AddDbContext<ApplicationDbContext>(opt =>
    opt.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>().AddDefaultTokenProviders();

// 2. Dual Authentication Setup (Cookie for MVC + JWT Bearer for REST Web API)
builder.Services.ConfigureApplicationCookie(opt => opt.LoginPath = "/Account/Login");
builder.Services.AddAuthentication()
    .AddJwtBearer(opt => {
        opt.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuer = true, ValidateAudience = true, ValidateLifetime = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
        };
    });

builder.Services.AddScoped<IAppUserRepository, AppUserRepository>();
builder.Services.AddControllersWithViews();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();
if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(); }

app.UseHttpsRedirection(); app.UseStaticFiles(); app.UseRouting();
app.UseAuthentication(); app.UseAuthorization();

// Data Seeding (Automatic Admin Account Initialization)
using (var scope = app.Services.CreateScope()) {
    var um = scope.ServiceProvider.GetRequiredService<UserManager<IdentityUser>>();
    var rm = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
    if (!await rm.RoleExistsAsync("Admin")) await rm.CreateAsync(new IdentityRole("Admin"));
    if (await um.FindByEmailAsync("admin@example.com") == null) {
        var admin = new IdentityUser { UserName = "admin@example.com", Email = "admin@example.com", EmailConfirmed = true };
        await um.CreateAsync(admin);
    }
}
```

> ជំហានទី ៦: EF Core Database Migrations

3. APPLY

Terminal

```
dotnet ef migrations add InitialCodeFirstMigration
dotnet ef database update
```

🔗 ជំហានទី ៧.១: Auth API Controller (JWT Login Endpoint)

6. CREATE

Controllers/Api/AuthApiController.cs

```
[Route("api/[controller]")]
[ApiController]
public class AuthApiController : ControllerBase
{
    private readonly UserManager<IdentityUser> _userManager;
    private readonly IConfiguration _config;

    public AuthApiController(UserManager<IdentityUser> userManager, IConfiguration config)
    {
        _userManager = userManager;
        _config = config;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginDto model)
    {
        var user = await _userManager.FindByEmailAsync(model.Email);
        if (user != null && await _userManager.CheckPasswordAsync(user, model.Password))
        {
            var claims = new List<Claim> {
                new Claim(ClaimTypes.Name, user.UserName!),
                new Claim(ClaimTypes.NameIdentifier, user.Id)
            };
            foreach (var role in await _userManager.GetRolesAsync(user))
            {
                claims.Add(new Claim(ClaimTypes.Role, role));
            }

            var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
            var token = new JwtSecurityToken(
                issuer: _config["Jwt:Issuer"], audience: _config["Jwt:Audience"],
                claims: claims, expires: DateTime.Now.AddHours(3),
                signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256)
            );

            return Ok(new TokenResponseDto {
                Token = new JwtSecurityTokenHandler().WriteToken(token),
                Expiration = token.ValidTo
            });
        }
        return Unauthorized("Email ឬ ពាក្យសម្ងាត់មិនត្រឹមត្រូវ");
    }
}
```

 ជំហានទី ៧.២: AppUser RESTful Web API Controller

6. CREATE

```
Controllers/Api/AppUserApiController.cs

[Route("api/[controller]")]
[ApiController]
[Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]
public class AppUserApiController : ControllerBase
{
    private readonly IAppUserRepository _userRepo;
    public AppUserApiController(IAppUserRepository userRepo) => _userRepo = userRepo;

    // GET: api/AppUserApi
    [HttpGet]
    public async Task<ActionResult<IEnumerable<AppUser>>> GetAll()
        => Ok(await _userRepo.GetAllAsync());

    // GET: api/AppUserApi/5
    [HttpGet("{id}")]
    public async Task<ActionResult<AppUser>> GetById(int id)
    {
        var user = await _userRepo.GetByIdAsync(id);
        return user == null ? NotFound() : Ok(user);
    }

    // POST: api/AppUserApi
    [HttpPost]
    [Authorize(Roles = "Admin", AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]
    public async Task<ActionResult> Create([FromBody] AppUser user)
    {
        if (!ModelState.IsValid) return BadRequest(ModelState);
        await _userRepo.InsertAsync(user);
        return CreatedAtAction(nameof(GetById), new { id = user.Id }, user);
    }

    // PUT: api/AppUserApi/5
    [HttpPut("{id}")]
    [Authorize(Roles = "Admin", AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]
    public async Task<IActionResult> Update(int id, [FromBody] AppUser user)
    {
        if (id != user.Id || !ModelState.IsValid) return BadRequest();
        await _userRepo.UpdateAsync(user);
        return NoContent();
    }

    // DELETE: api/AppUserApi/5
    [HttpDelete("{id}")]
    [Authorize(Roles = "Admin", AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]
    public async Task<IActionResult> Delete(int id)
    {
        if (await _userRepo.GetByIdAsync(id) == null) return NotFound();
        await _userRepo.DeleteAsync(id);
        return NoContent();
    }
}
```

🖥 ជំហានទី ៨: MVC Controllers

6. CREATE

៨.១ File: `Controllers/AccountController.cs` (MVC Login/Logout)

```
public class AccountController : Controller
{
    private readonly SignInManager<IdentityUser> _signInManager;
    public AccountController(SignInManager<IdentityUser> signInManager) => _signInManager = signInManager;

    [HttpGet, AllowAnonymous]
    public IActionResult Login(string? returnUrl = null) { ViewData["ReturnUrl"] = returnUrl; return View(); }

    [HttpPost, AllowAnonymous, ValidateAntiForgeryToken]
    public async Task<IActionResult> Login(LoginDto model, string? returnUrl = null)
    {
        if (ModelState.IsValid) {
            var res = await _signInManager.PasswordSignInAsync(model.Email, model.Password, model.RememberMe, false);
            if (res.Succeeded) return RedirectToLocal(returnUrl);
            ModelState.AddModelError("", "កាកប៉ុនប៉ងចូលប្រើប្រាស់មិនត្រឹមត្រូវ!");
        }
        return View(model);
    }

    [HttpPost, ValidateAntiForgeryToken]
    public async Task<IActionResult> Logout() { await _signInManager.SignOutAsync(); return RedirectToAction("Login"); }

    private IActionResult RedirectToLocal(string? returnUrl)
    => (!string.IsNullOrEmpty(returnUrl) && Url.IsLocalUrl(returnUrl)) ? Redirect(returnUrl) : RedirectToAction("Login");
}
```

៨.២ File: `Controllers/AppUserController.cs` (MVC Views CRUD)

```
[Authorize]
public class AppUserController : Controller
{
    private readonly IAppUserRepository _repo;
    public AppUserController(IAppUserRepository repo) => _repo = repo;

    public async Task<IActionResult> Index() => View(await _repo.GetAllAsync());

    [Authorize(Roles = "Admin")] public IActionResult Create() => View();

    [HttpPost, Authorize(Roles = "Admin"), ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(AppUser user) {
        if (ModelState.IsValid) { await _repo.InsertAsync(user); return RedirectToAction(nameof(Index)); }
        return View(user);
    }

    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> Edit(int id) => View(await _repo.GetByIdAsync(id));

    [HttpPost, Authorize(Roles = "Admin"), ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(AppUser user) {
        if (ModelState.IsValid) { await _repo.UpdateAsync(user); return RedirectToAction(nameof(Index)); }
        return View(user);
    }

    [HttpPost, Authorize(Roles = "Admin"), ValidateAntiForgeryToken]
    public async Task<IActionResult> Delete(int id) { await _repo.DeleteAsync(id); return RedirectToAction(nameof(Index)); }
}
```


📁 ជំហានទី ៩: Razor Views (.cshtml) សម្រាប់ MVC UI

6. CREATE

៩.១ File: `Views/Account/Login.cshtml`

```
<@model MyCodeFirstProject.Models.DTOs.LoginDto>
<div class="row justify-content-center mt-4">
  <div class="col-md-4">
    <h3 class="text-center">ចូលប្រើប្រាស់ប្រព័ន្ធ (MVC)</h3>
    <form asp-action="Login" method="post">
      <div class="mb-3">
        <label asp-for="Email"></label>
        <input asp-for="Email" class="form-control" />
      </div>
      <div class="mb-3">
        <label asp-for="Password"></label>
        <input asp-for="Password" class="form-control" />
      </div>
      <button type="submit" class="btn btn-primary w-100">ចូលប្រើ</button>
    </form>
  </div>
</div>
```

៩.២ File: `Views/AppUser/Index.cshtml`

```
<@model IEnumerable<MyCodeFirstProject.Models.AppUser>>
<@inject SignInManager<IdentityUser> SignInManager>

<div class="d-flex justify-content-between my-3">
  <h2>បញ្ជីអ្នកប្រើប្រាស់ (EF Core Code-First)</h2>
  @if (SignInManager.IsSignedIn(User) && User.IsInRole("Admin")) {
    <a asp-action="Create" class="btn btn-primary">បង្កើតថ្មី</a>
  }
</div>

<table class="table table-bordered table-striped">
  <thead class="table-dark">
    <tr>
      <th>ឈ្មោះ</th>
      <th>អាសយដ្ឋាន Email</th>
      <th>ស្ថានភាព</th>
      <th>សកម្មភាព</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var u in Model) {
      <tr>
        <td>@u.FullName</td>
        <td>@u.Email</td>
        <td>@(u.IsActive ? "សកម្ម" : "អសកម្ម")</td>
        <td>
          @if (User.IsInRole("Admin")) {
            <a asp-action="Edit" asp-route-id="@u.Id" class="btn btn-sm btn-warning">កែប្រែ</a>
            <form asp-action="Delete" asp-route-id="@u.Id" method="post" class="d-inline">
              <button type="submit" class="btn btn-sm btn-danger" onclick="return confirm('លុប?')">លុប</button>
            </form>
          }
        </td>
      </tr>
    }
  </tbody>
</table>
```

ជំហានទី ១០: ការវិភាគប្រៀបធៀប Cookie-Auth VS JWT-Bearer Auth

4. ANALYZE

ការយល់ដឹងពីយន្តការសុវត្ថិភាពទាំងពីរគឺជារឿងសំខាន់បំផុតក្នុងការកសាងប្រព័ន្ធ Enterprise Hybrid Application (MVC + Web API) ៖

លក្ខណៈវិនិច្ឆ័យ	Cookie-Based Auth (MVC)	JWT Bearer Auth (REST Web API)
ទីតាំងរក្សាទុក (Storage)	HTTP Cookie ក្នុង Browser (HttpOnly, Secure)	Client-side (LocalStorage, SessionStorage, Memory)
State Management	Stateful / Session-based: Server រក្សា Session	Stateless: Server មិនរក្សា Session ទេ (Token-self contained)
Header Format	បញ្ជូនដោយស្វ័យប្រវត្តតាម Cookie: .AspNetCore.Cookies=...	បញ្ជូនតាម Custom Header Authorization: Bearer <Token>
ហានិភ័យសុវត្ថិភាព (Risks)	ងាយរងគ្រោះនឹង CSRF Attack (ត្រូវការ Anti-Forgery Token)	ងាយរងគ្រោះនឹង XSS Attack ប្រសិនបើស្រាយ Token ខុសវិធី
ការប្រើប្រាស់សក្តានុពល	Monolithic Web Applications, Server-Side Rendering (Razor)	Mobile Apps, SPA Frameworks (React, Angular, Vue), Microservices



យុទ្ធសាស្ត្រការពារសុវត្ថិភាពទីល្អបំផុត (Security Best Practices)

1. Token Expiration: ត្រូវកំណត់អាយុកាល Token ឱ្យខ្លី (ឧទាហរណ៍៖ ៣០នាទី ទៅ ៣ម៉ោង) ដើម្បីកាត់បន្ថយហានិភ័យពេលលេចធ្លាយ Token។
2. Secret Key Strength: Key ដែលប្រើក្នុង `appsettings.json` ត្រូវតែមានប្រវែងយ៉ាងតិច 32 characters (256 bits) ដើម្បីការពារ Brute Force Attack។
3. Validate Anti-Forgery: ប្រើប្រាស់ [ValidateAntiForgeryToken] លើគ្រប់ POST/PUT/DELETE Actions ក្នុង MVC Controllers។

សំណួរវិភាគមេរៀន (Bloom's Taxonomy Quiz) - ភាគ ១

១. សំណួរ ខុស/ត្រូវ (True/False)

1. REMEMBER

1. EF Core Code-First អនុញ្ញាតឱ្យយើងបង្កើត Database ស្វ័យប្រវត្តិចេញពី C# Entity Models ដោយមិនបាច់សរសេរ SQL Command។
2. JWT (JSON Web Token) ត្រូវការរក្សាទុក Session State នៅលើ Web Server ជានិច្ចដើម្បីផ្ទៀងផ្ទាត់ Identity។
3. 'IdentityDbContext' គឺជា Class ដែលមានស្រាប់ក្នុង ASP.NET Core Identity សម្រាប់បង្កើតតារាង User, Role, និង Claims។
4. កូដ '[Authorize(AuthenticationSchemes = JwtBearerDefaults.AuthenticationScheme)]' ប្រើសម្រាប់ការការពារ API Endpoints ដោយតម្រូវឱ្យមាន JWT Token។
5. status code 200 OK គឺជា Response តែមួយគត់ដែលបញ្ជាក់ថា API Auth login ទទួលបានជោគជ័យ។

២. សំណួរ ពហុជ្រើសរើស (MCQs)

3. APPLY

1. តើ Package មួយណាដែលចាំបាច់បំផុតសម្រាប់ការសរសេរ Migration និងបញ្ជា 'dotnet ef database update' ?
A. A. Microsoft.EntityFrameworkCore.SqlServer
B. B. Microsoft.EntityFrameworkCore.Tools
C. C. Swashbuckle.AspNetCore
D. D. Microsoft.AspNetCore.Authentication.JwtBearer
2. តើ Header មួយណាដែលត្រូវប្រើប្រាស់ដើម្បីបញ្ជូន JWT Token ទៅកាន់ Protected Web API ?
A. A. Content-Type: application/json
B. B. Authorization: Bearer <token>
C. C. Cookie: .AspNetCore.Identity
D. D. Accept: token/jwt
3. ដើម្បីសរសេរ Data Seeding ឱ្យបង្កើត Admin Account ដោយស្វ័យប្រវត្តិ ពេល Start App តើយើងត្រូវប្រើ Service មួយណា ?
A. A. SignInManager<IdentityUser>
B. B. UserManager<IdentityUser>
C. C. DbContextOptions
D. D. IAppUserRepository

៣. សំណួរផ្គូផ្គង (Matching)

2. UNDERSTAND

សេចក្តីណែនាំ: សូមផ្គូផ្គងពាក្យបច្ចេកទេសនៅជួរឈរខាងឆ្វេង ទៅនឹងការពន្យល់នៅជួរឈរខាងស្តាំឱ្យបានត្រឹមត្រូវ។

១. SymmetricSecurityKey	A. ចុះឈ្មោះ: Service ឱ្យបង្កើត Instance ថ្មីមួយក្នុង 1 HTTP Request Cycle។
២. AddDbContext	B. បញ្ចូល Claim ប្រភេទ Role ទៅក្នុង JWT Token ដើម្បីប្រើប្រាស់ Role-Based Authorization។
៣. [ValidateAntiForgeryToken]	C. ប្រើប្រាស់ Secret Key សម្រាប់ Encrypt និង Validate ហត្ថលេខាឌីជីថលរបស់ JWT Token។
៤. ClaimTypes.Role	D. ចុះឈ្មោះ: Database Context ទៅក្នុង Dependency Injection Container។
៥. AddScoped	E. ការការពារការប្រហារ Cross-Site Request Forgery លើ MVC Forms។

៤. សំណួរបកស្រាយគម្រិតខ្ពស់ (Critical Thinking & Essay Questions)

5. EVALUATE

- ANALYZE** ចូរពន្យល់ពីមូលហេតុដែលប្រព័ន្ធ Hybrid (MVC + Web API) ចាំបាច់ត្រូវប្រើប្រាស់ Dual Authentication Scheme (Cookie-Based + JWT Bearer) ក្នុង `Program.cs`។
- EVALUATE** ប្រៀបធៀបគុណសម្បត្តិ និងគុណវិបត្តិនៃការរក្សាទុក JWT Token នៅក្នុង `LocalStorage` ប្រៀបធៀបនឹង `HttpOnly Cookie` លើ Client-Side Application។

🔑 អ្នកត្រូវដឹង (Answer Keys & Solutions)

១. ចម្លើយ ខុស/ត្រូវ (True/False)

១	២	៣	៤	៥
ត្រូវ	ខុស	ត្រូវ	ត្រូវ	ត្រូវ

២. ចម្លើយ ពហុជ្រើសរើស (MCQs)

១	២	៣
B	B	B

៣. ចម្លើយ ផ្គូផ្គង (Matching Keys)

១	២	៣	៤	៥
C	D	E	B	A

៤. ចម្លើយ សំណួរបកស្រាយ (សង្ខេប)

- ១. មូលហេតុប្រើ Dual Auth Scheme: ដោយសារ MVC ដំណើរការតាមបែប Server-Side Rendering រក្សា State តាម Cookie និង Redirect ទៅ Login Page ពេលអត់ Auth, ចំណែក Web API ដំណើរការតាមបែប Stateless (Client-Server Decoupled) ទទួល JSON តាម HTTP Header Authorization Bearer និងបញ្ចេញ 401 Unauthorized ពេលអត់ Token។ ការដាក់ Dual Auth ធ្វើឱ្យ Framework ស្គាល់ Authenticate Scheme ទាំងពីរក្នុង App តែមួយ។
- ២. LocalStorage VS HttpOnly Cookie: LocalStorage ងាយស្រួលប្រើប្រាស់ជាមួយ JS Fetch/Axios ប៉ុន្តែប្រឈមមុខនឹង XSS Attack ខ្ពស់ប្រសិន code មានចន្លោះប្រហោង។ ចំណែក HttpOnly Cookie ការពារ XSS បានល្អ (JS មើលមិនឃើញ) ប៉ុន្តែប្រឈមមុខនឹង CSRF Attack និងពិបាកកំណត់សិទ្ធិប្រើប្រាស់ Cross-Domain Mobile APIs។

🔗 ជំហានទី ១១: ការធ្វើតេស្ត និងការផ្ទៀងផ្ទាត់ប្រព័ន្ធ (Testing)

5. EVALUATE



១. តេស្តតាមរយៈ Web Browser (MVC Razor Views)

1. Run គម្រោងដោយប្រើបញ្ជា `dotnet run` ក្នុង Terminal។
2. បើក Browser ទៅកាន់ URL: `https://localhost:7123/`
3. ប្រព័ន្ធនឹង Redirect ដោយស្វ័យប្រវត្តិទៅកាន់ទំព័រ Login /Account/Login
4. ចូលប្រើប្រាស់ដោយប្រើគណនី Admin ដែលបាន Seed ក្នុង `Program.cs`:
 - Email: `admin@example.com`
 - Password: `Admin123!`



២. តេស្ត RESTful Web API តាមរយៈ Swagger UI / Postman

1. បើក Swagger UI តាមរយៈ URL: `https://localhost:7123/swagger`
2. ជំហានទី ១ (Get JWT Token): ហៅ API POST `/api/AuthApi/login` ដោយផ្ញើ JSON Payload:

```
{ "email": "admin@example.com", "password": "Admin123!" }
```
3. ចម្លងអត្ថបទ Token ដែលទទួលបានពី Response Body។
4. ជំហានទី ២ (Authorize): ចុចប៊ូតុង **Authorize** នៅខាងលើស្តាំក្នុង Swagger រួចវាយបញ្ចូល៖ **Bearer <YOUR_JWT_TOKEN>**
5. ជំហានទី ៣ (CRUD APIs): ឥឡូវអ្នកអាចធ្វើតេស្ត API Endpoints ទាំងអស់ក្នុង `AppUserApiController` (GET, POST, PUT, DELETE) បានដោយរលូន។

★ Enterprise Pro Tip: SweetAlert2 Alert Box Integration

សម្រាប់ការបង្ហាញប្រអប់សារលេចស្អាតពេលធ្វើការជាមួយ API Client គេនិយមប្រើ `Swal.fire("Success", "Authenticated via JWT Token", "success");`



អបអរសាទរ! លោកអ្នកបានបញ្ចប់ការសិក្សាពី ប្រព័ន្ធ Dual-Auth ដោយជោគជ័យ។

ពេលនេះអ្នកបានទទួលបានចំណេះដឹងនិងជំនាញក្នុងការសាងសង់ប្រព័ន្ធ **ASP.NET Core Web Application** ដែលប្រើប្រាស់ **EF Core Code-First** ក្នុងការគ្រប់គ្រង Database, អនុវត្ត **Repository Pattern**, ព្រមទាំងរៀបចំប្រព័ន្ធសុវត្ថិភាព **Dual Authentication (Cookie-Based Auth + JWT Token Auth)** យ៉ាងមានប្រសិទ្ធភាព។

🏆 សមិទ្ធផលបច្ចេកទេសដែលអ្នកទទួលបាន៖

- ការបង្កើត Tables និង Database ដោយស្វ័យប្រវត្តិជាមួយ EF Core Code-First Migrations។
- ការការពារទំព័រ MVC Razor Views ដោយប្រើ Cookie Authentication & Role Authorization។
- ការការពារ Endpoints របស់ RESTful Web API ដោយប្រើប្រាស់ JWT (JSON Web Token) Bearer Header។
- ការអនុវត្ត Data Seeding ដើមគ្រោសប្រាប់បង្កើត Admin User និង Master Roles ក្នុងប្រព័ន្ធ។

*** END OF CODE-FIRST DUAL-AUTH GUIDE ***